



FOUR-COURSE SYSTEM

COURSE 3 - DATA ENGINEERING 2026

C3-020 Data Ingestion & Interfaces - Beginner-First Lesson Book

14 lessons | source evidence -> defensive interfaces -> incremental/CDC -> replay-safe operations

Artifact	C3_020_DATA_INGESTION_INTERFACES_BEGINNER_FIRST_LESSON_BOOK_RC1
Status	2026 - LEARNER EDITION
Teaching standard	M01 beginner-first teacher loop. Videos are visual companions; the book remains sufficient fallback.
C2B boundary	Bridge foundations may be assumed, but C3 engineering behavior is demonstrated, broken, repaired, transferred and validated.
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

Course rule: preserve what arrived -> validate interpretation -> quarantine ambiguity -> publish durably -> commit state last -> prove replay safety. A green job is never sufficient evidence by itself.

This book is your teacher

Use this book as the primary teacher. External videos and official documentation are visual or current-behavior companions, never hidden prerequisites.

The locked beginner-first lesson contract

- | | |
|-----------------------------------|---|
| 01. Why it matters | 09. Why it worked |
| 02. What you learn | 10. Common beginner mistakes |
| 03. Prerequisite / starting state | 11. If stuck / recovery |
| 04. Picture it in your head | 12. Small independent task |
| 05. New words before use | 13. Validate |
| 06. Useful video/source | 14. Teach it back |
| 07. Follow with me once | 15. Protected review + changed retry |
| 08. Expected result | 16. Evidence/mastery + exact next route |

Rules that protect your learning

- Save a genuine attempt before opening a protected review.
- After review, close it and solve a changed retry - do not copy the reviewed answer.
- A command that runs is not mastery: validate grain, keys, counts, state, failure/recovery and evidence.
- If a new engineering concept appears, this book explains it from first principles even when Bridge skills are assumed.
- When stuck, repair only the named weak competency; do not restart the whole phase.

Contents / Index

Actual page numbers after the master-audit repair. Mentor routes use these same pages.

Front matter	Page
This book is your teacher	2
Contents / Index	3
Module map + practice/review/Gate route	4
Phase-specific study notes	5
Lessons	Start
C3-01-L01 Ingestion architecture: source -> landing -> staging	6
C3-01-L02 CSV ingestion and defensive parsing	9
C3-01-L03 JSON and nested-record ingestion	12
C3-01-L04 Parquet ingestion and metadata inspection	15
C3-01-L05 REST APIs: requests, status codes and contracts	18
C3-01-L06 API authentication, pagination and rate limits	21
C3-01-L07 Retries, backoff, jitter and transient failures	24
C3-01-L08 Database extraction and safe query boundaries	27
C3-01-L09 Historical batch loading and landing metadata	30
C3-01-L10 Incremental loads and high-water marks	33
C3-01-L11 CDC concepts and change-event semantics	36
C3-01-L12 Schema drift, compatibility, temporal/time-zone correctness and quarantine	39
C3-01-L13 Checkpoint/state management and replay safety	42
C3-01-L14 Ingestion testing, reconciliation and backfills	45
Mastery checklist + revision quick reference	48

Module map + protected learning route

Data Ingestion & Interfaces

- 01 Prerequisite / prior-phase Gate
- 02 Lesson mental model + vocabulary
- 03 Follow guided example once
- 04 Matching independent practice
- 05 Save genuine attempt
- 06 Protected review unlocks
- 07 Close review + changed retry
- 08 Validate + explain back
- 09 Master lesson + save evidence
- 10 Mini-Lab / integrated drill
- 11 Protected Gate A
- 12 If needed: protected review -> targeted repair -> changed micro-proof
- 13 Explicitly assign fresh Gate B/C only after repair evidence
- 14 Phase PASS -> next phase

What counts as mastery

- You can rebuild the competency without the lesson/review open.
- Your validation catches a plausible wrong result, not only syntax failure.
- You can state one failure mode and the recovery step.
- Your evidence names the input/output or artifact and the independent check used.

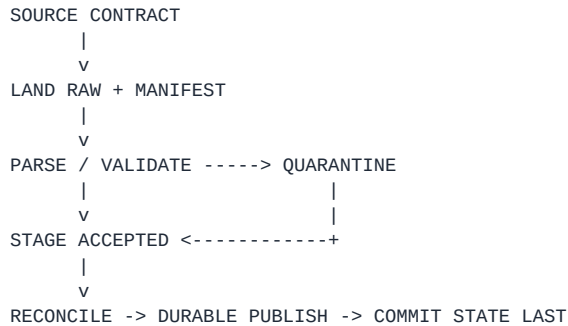
Revision rule

Revise from evidence: weak validation, weak recovery or weak explanation sends you back only to the matching lesson/repair route. Completion percentage never overrides a failed Gate.

How this phase fits after Course 2B

Course 2B gave you competent analyst automation, API/file handling and data-quality reasoning. C3-01 does not reteach those as analyst tasks. It turns them into engineering boundaries that must survive malformed inputs, pagination, retries, ties, schema drift, restarts and historical replay.

THE INGESTION CORRECTNESS LOOP



TOOL INSTALL POLICY

L01-L03 use the Python standard library. At L04, install PyArrow only if the active C3 venv cannot import it. At L05, install Requests only if missing. sqlite3 is in Python; pytest was introduced in C3-00. Kafka, Debezium server, Airflow, Spark and cloud runtimes are NOT installed in C3-01.

MASTERY LOOP

Read -> optional/recommended visual -> follow once -> expected result -> break/fix -> independent changed version -> validate independently -> explain back -> freeze evidence -> open protected review -> close it -> changed retry -> master.

No external video is required to pass. Current official references control tool-sensitive behavior; the lesson book supplies the complete teaching path.



C3-01-L01 - Ingestion architecture: source -> landing -> staging

Focused effort: 2.0 h | Matching practice P01 | Overlap: deeper engineering progression

WHY IT MATTERS

If a cleaned table is wrong tomorrow, you need to prove what actually arrived today. An engineer therefore separates evidence from interpretation.

WHAT YOU LEARN

Design a source contract; preserve immutable landing evidence; create a controlled staging boundary; reconcile source, accepted and quarantined records.

PREREQUISITE / BRIDGE BOUNDARY

C2B B02 file/API familiarity and B04 quality thinking. No prior C3-01 lesson.

Bridge taught handling data; this lesson changes the center of gravity to preserving source evidence and operating an ingestion boundary.

PICTURE IT IN YOUR HEAD

Imagine a parcel depot. The loading dock photographs every sealed package exactly as received (landing). Only after that does a sorting desk open, inspect and route parcels (staging). If the sorting desk rewrites the original package, investigation becomes impossible.

NEW WORDS BEFORE WE USE THEM

source contract - Declared producer/source, object/query, expected shape, ownership and freshness rules.

landing - Raw evidence of what arrived, plus run metadata.

staging - Controlled parsing/normalization area before trusted publication.

manifest - Small record describing run ID, source, time, count, fingerprint and paths.

fingerprint - Hash or other stable identity used to detect changed bytes.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Course-built ingestion contract

Local lesson book

Purpose: Source -> landing -> staging boundary

Useful segment: Whole lesson

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.



C3-01-L01 - Follow with me once

FOLLOW WITH ME ONCE

- Create `work/L01/landing/run\_001/` and `work/L01/staging/run\_001/`.
- Copy one source file into landing without editing it.
- Create manifest.json with run_id, source_name, extracted_at_utc, raw_path, byte_count, record_count and sha256.
- Only after landing is durable, parse/validate into staging.
- Record `source\_rows = accepted\_rows + quarantine\_rows` as an independent reconciliation.

TYPE / RUN

```
python - <<'PY'
from pathlib import Path
import hashlib, json
p=Path('Practice/data/customers_messy.csv')
raw=p.read_bytes()
print({'bytes':len(raw), 'sha256':hashlib.sha256(raw).hexdigest()[:16]})
PY
```

EXPECTED RESULT

A byte count and stable hash prefix print. Re-running without changing the file produces the same hash.

WHY IT WORKED

The hash identifies source bytes independently of whatever your parser later decides. Landing and staging can now disagree without destroying evidence.

COMMON BEGINNER MISTAKES

- Writing cleaned output over the source file.
- Calling staging "raw" after types/columns were already changed.
- Using only a filename such as latest.csv with no run/window identity.
- Recording accepted count but not quarantine count.

RECOVERY IF STUCK

If you already edited the only copy, stop. Re-acquire the source if possible, create a new immutable landing run, and document that the earlier run lacks trustworthy raw evidence.



C3-01-L01 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Use `orders\_nested.json` instead. Define a fresh source contract and manifest without copying CSV-specific fields.

VALIDATION - DO NOT SKIP

Prove raw hash/path exists; accepted + quarantine = source records; staging path is different from landing; manifest has UTC run time.

ARCHITECTURE / PRODUCTION REASONING

Object storage often gives cheap immutable or versioned landing. At scale, retention, encryption, access control and lifecycle rules become part of the contract; the conceptual boundary does not change.

TEACH IT BACK

In 60-90 seconds explain why landing is evidence, staging is interpretation, and what you would inspect during a dispute.

PROTECTED REVIEW + CHANGED RETRY

Freeze P01 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: After review, change source type and one contract field. Rebuild from blank without looking at the reviewed attempt.

EVIDENCE / MASTERY

architecture note + manifest + hash/count output + reconciliation + changed-transfer explanation.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L02 - CSV ingestion and defensive parsing

Focused effort: 2.5 h | Matching practice P02 | Overlap: deeper engineering progression

WHY IT MATTERS

A malformed CSV often produces plausible-but-shifted columns rather than a loud crash. Silent corruption is more dangerous than failure.

WHAT YOU LEARN

Use a real parser, explicit newline/encoding, header contract, deliberate type conversion, quarantine reasons and row reconciliation.

PREREQUISITE / BRIDGE BOUNDARY

L01. Bridge B02 already used CSV: this is deeper engineering treatment, not a repeat.

Assume you know how to read a CSV. Now you must defend dialect, headers, quoting, blanks, types and quarantine behavior under messy input.

PICTURE IT IN YOUR HEAD

A CSV is a stack of forms, but commas can appear inside quoted answers. ``split(',')`` is like cutting every form wherever you see a comma, including inside a person's name. A CSV parser knows the quoting rules.

NEW WORDS BEFORE WE USE THEM

dialect - Delimiter/quote/escape conventions used by a CSV producer.

header contract - Required column names and allowed extras.

quarantine - Preserved rejected records plus reasons; not silent deletion.

coercion - Converting a value into another type, sometimes dangerously.

row reconciliation - Proof that every source record is accounted for.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Python 3.14.7 csv

<https://docs.python.org/3.14/library/csv.html>

Purpose: DictReader, newline handling, dialects

Useful segment: Reader/DictReader + newline guidance

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.



C3-01-L02 - Follow with me once

FOLLOW WITH ME ONCE

- Open `Practice/data/customers\_messy.csv` as raw evidence first.
- Use `csv.DictReader` with `encoding="utf-8"` and `newline=""`.
- Validate required headers before rows.
- Parse `credit\_limit` explicitly; do not turn blank into zero unless the source contract says so.
- Write valid rows to accepted.csv and invalid/ambiguous rows to quarantine.csv with source_row + reason.

TYPE / RUN

```
import csv
with open('Practice/data/customers_messy.csv', encoding='utf-8', newline='') as f:
    reader=csv.DictReader(f)
    print(reader.fieldnames)
    for i,row in zip(range(3), reader):
        print(row)
```

EXPECTED RESULT

The quoted comma in a name remains inside one `name` value; headers print once and rows are dictionaries rather than manually split lists.

WHY IT WORKED

`csv.DictReader` applies CSV quoting/newline rules and exposes fields by name, so you can validate schema and convert types deliberately.

COMMON BEGINNER MISTAKES

- Using `line.split(",")`.
- Assuming blank numeric value means 0.
- Dropping bad rows without raw row + reason.
- Trusting parsed row count without source accounting.

RECOVERY IF STUCK

If widths or values shift, inspect `reader.fieldnames`, `reader.line\_num`, the raw line and dialect assumptions. Do not "fix" output values before preserving the bad source row.



C3-01-L02 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Create a fresh semicolon-delimited file with one quoted semicolon and one invalid number. Configure the parser deliberately and quarantine only the invalid row.

VALIDATION - DO NOT SKIP

accepted + quarantine = source data rows; required headers present; keys unique where required; two manual raw-to-output comparisons; quoted delimiter preserved.

ARCHITECTURE / PRODUCTION REASONING

Large ingestion services add contract/version tracking and producer-specific dialect configuration, but parser correctness and quarantine accounting remain the same.

TEACH IT BACK

Explain why "the file opened successfully" does not prove a CSV is correct.

PROTECTED REVIEW + CHANGED RETRY

Freeze P02 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Use a different delimiter/blank pattern and rebuild without opening the review.

EVIDENCE / MASTERY

parser code + accepted/quarantine + counts + raw comparison + one failure/recovery note.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L03 - JSON and nested-record ingestion

Focused effort: 2.5 h | Matching practice P03 | Overlap: deeper engineering progression

WHY IT MATTERS

Conveniently flattening nested arrays can multiply parent facts. The pipeline can run and revenue can still be wrong.

WHAT YOU LEARN

Declare grains before flattening; separate parent and child outputs; preserve identifiers/run lineage; reconcile child-array counts.

PREREQUISITE / BRIDGE BOUNDARY

L02. Bridge B02 JSON familiarity assumed.

You have seen JSON before; this lesson adds explicit output grain, parent/child cardinality controls and lineage.

PICTURE IT IN YOUR HEAD

An order is a folder and its items are pages inside it. Copying the order total onto every page makes a three-item order look three times as large if someone sums the pages.

NEW WORDS BEFORE WE USE THEM

nested object - Object inside another JSON object.

array - Ordered collection that can contain zero or many child records.

grain - What one output row represents.

cardinality - How many parent/child records can relate.

lineage key - Identifier that lets an output record be traced back to source/run.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Python 3.14.7 json

<https://docs.python.org/3.14/library/json.html>

Purpose: Nested JSON parsing/serialization

Useful segment: load/loads + encoder/decoder behavior

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.



C3-01-L03 - Follow with me once

FOLLOW WITH ME ONCE

- Open `orders\_nested.json` and state source grain before code.
- Define `orders.jsonl` at one row per order and `order\_items.jsonl` at one row per item.
- Emit one parent record once; loop `items` only for child output.
- Preserve order_id and run/batch ID in both outputs.
- Compute parent rows and sum of source item-array lengths as independent controls.

TYPE / RUN

```
import json
obj=json.load(open('Practice/data/orders_nested.json',encoding='utf-8'))
orders=obj['orders'] if isinstance(obj,dict) else obj
print('orders',len(orders))
print('items',sum(len(o.get('items',[])) for o in orders))
```

EXPECTED RESULT

Two counts print: number of parent orders and total child items. Your normalized outputs should later reconcile to exactly those grains.

WHY IT WORKED

The control counts come directly from source structure before flattening, so they can detect accidental row multiplication.

COMMON BEGINNER MISTAKES

- Flattening parent + child into one table then summing parent measures.
- Losing parent key on child rows.
- Treating missing array and empty array as identical without contract.
- Not recording source/run identity.

RECOVERY IF STUCK

If totals inflate, stop using the flattened output. Restate parent and child grain on paper, regenerate separate outputs, then reconcile before business aggregation.



C3-01-L03 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Add a nested `shipping` object and preserve its fields without changing item grain; then add an order with an empty items list.

VALIDATION - DO NOT SKIP

parent output count = source orders; child count = total source items; each child has parent key; parent total reconciles once per order.

ARCHITECTURE / PRODUCTION REASONING

Production normalizers often create multiple typed tables/events from one source document. Grain and lineage are architectural contracts, not pandas tricks.

TEACH IT BACK

Explain why nested JSON is primarily a grain/cardinality problem, not just a syntax problem.

PROTECTED REVIEW + CHANGED RETRY

Freeze P03 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Normalize a fresh customer->addresses document into customer and address grains.

EVIDENCE / MASTERY

two outputs + grain definitions + lineage fields + source-derived row controls + changed transfer.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L04 - Parquet ingestion and metadata inspection

Focused effort: 2.5 h | Matching practice P04 | Overlap: deeper engineering progression

WHY IT MATTERS

Parquet is not "a CSV with another extension." Its schema, row groups and columnar layout influence ingestion, compatibility and later distributed performance.

WHAT YOU LEARN

Install/verify PyArrow; generate real Parquet; inspect schema/metadata/row groups; project only required columns; reconcile metadata to read results.

PREREQUISITE / BRIDGE BOUNDARY

L03 and the C3 project venv from C3-00. PyArrow is installed here at first use.

New engineering/storage skill. Bridge does not need to have taught Parquet internals.

PICTURE IT IN YOUR HEAD

Think of a library where each column is shelved together and the catalogue tells you how many sections/records exist. You can inspect the catalogue and fetch selected shelves instead of opening every page.

NEW WORDS BEFORE WE USE THEM

columnar - Values stored by column rather than row-oriented text layout.

row group - Chunk of rows with its own metadata/statistics.

projection - Reading only requested columns.

schema - Field names and physical/logical types.

metadata - Structural information stored with the Parquet file.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Apache Arrow Python Parquet

<https://arrow.apache.org/docs/python/parquet.html>

Purpose: ParquetFile metadata, row groups, projected reads

Useful segment: Parquet reading + metadata

RECOMMENDED | Mathew K Analytics - Comprehensive PyArrow Training

<https://www.youtube.com/watch?v=4NdhT9JXWTo>

Purpose: Visualize real Parquet read/write and schema inspection

Useful segment: 06:09-09:40

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.

C3-01-L04 - Follow with me once

FOLLOW WITH ME ONCE

- Activate the C3 env. First check ``python -c "import pyarrow; print(pyarrow.__version__)"``.
- If missing, install only now: ``python -m pip install pyarrow``. Run the version check again.
- In ``Practice/starter_files/P04_parquet``, run ``make_parquet.py`` to create an actual ``.parquet``.
- Use ``pyarrow.parquet.ParquetFile`` to inspect metadata, schema and row groups before reading data.
- Read only ``device_id`` and ``event_ts``; compare projected row count with metadata row count.

TYPE / RUN

```
python -c "import pyarrow; print(pyarrow.__version__)"
python Practice/starter_files/P04_parquet/make_parquet.py
python Practice/starter_files/P04_parquet/inspect_TODO.py
```

EXPECTED RESULT

The version command prints a PyArrow version. A real Parquet file is created. Metadata and projected read report the same record count.

WHY IT WORKED

Parquet embeds schema and row-group metadata. PyArrow can inspect that structure and perform column projection without pretending the file is opaque row text.

COMMON BEGINNER MISTAKES

- Renaming CSV to ``.parquet``.
- Loading every column before checking schema.
- Assuming "Parquet" guarantees compatible types across files.
- Claiming runtime evidence if PyArrow never ran.

RECOVERY IF STUCK

If import fails, confirm the active venv and ``python -m pip show pyarrow``. If installation is blocked, record the blocker; do not fabricate Parquet evidence. If schema differs, inspect metadata before coercing.



C3-01-L04 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Generate a second real Parquet file with one additional nullable column. Compare schemas and decide whether the change is compatible.

VALIDATION - DO NOT SKIP

PyArrow version recorded; file opens as Parquet; metadata row count = projected row count; projection contains exactly requested columns; schema difference explained.

ARCHITECTURE / PRODUCTION REASONING

Parquet metadata later supports partition/file pruning and schema evolution decisions. This lesson is about correct ingestion evidence, not premature optimization.

TEACH IT BACK

Explain the difference between file format, schema metadata, row groups and projection.

PROTECTED REVIEW + CHANGED RETRY

Freeze P04 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Build a new Parquet from a changed CSV and inspect one different two-column projection.

EVIDENCE / MASTERY

install/verify output + real Parquet + metadata log + projection + row-count control + changed transfer.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L05 - REST APIs: requests, status codes and contracts

Focused effort: 2.5 h | Matching practice P05 | Overlap: deeper engineering progression

WHY IT MATTERS

HTTP 200 means the server returned a successful HTTP response. It does not prove the payload is the right media type, shape, completeness or business data.

WHAT YOU LEARN

Define method/endpoint/auth/params/timeout/status/media type/schema; make a bounded call; validate transport and content; sanitize logs.

PREREQUISITE / BRIDGE BOUNDARY

L01 and Bridge B02 API basics. Install Requests only if the C3 venv cannot import it.

Bridge taught calling APIs; C3 requires an explicit transport/content/schema contract and safe failure behavior.

PICTURE IT IN YOUR HEAD

A courier saying "delivery successful" proves a box reached your door, not that it contains the right product. HTTP status is delivery status; payload validation checks what is inside.

NEW WORDS BEFORE WE USE THEM

endpoint - Specific API URL/resource boundary.

timeout - Maximum wait rule for network operations.

status code - HTTP-level result such as 200, 401, 404, 429, 500.

media type - Declared response content type such as application/json.

contract validation - Checking required response behavior and fields before accepting data.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Requests Quickstart

<https://requests.readthedocs.io/en/latest/user/quickstart/>

Purpose: timeouts, response validation, status handling

Useful segment: Timeouts + response content + raise_for_status

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.

C3-01-L05 - Follow with me once

FOLLOW WITH ME ONCE

- Start the local mock API from P05_P07_mock_api.
- Check Requests: ``python -c "import requests; print(requests.__version__)"``; install only if missing.
- Call ``/health`` with a timeout and validate 200 + JSON.
- Put the supplied fake token in environment, not source; call ``/profile`` with Authorization header.
- Call without token once to capture expected 401 failure. Sanitize token from logs.

TYPE / RUN

```
# terminal 1
python Practice/starter_files/P05_P07_mock_api/mock_api.py
# terminal 2
python -c "import requests; print(requests.get('http://127.0.0.1:18765/health',
timeout=3).json())"
```

EXPECTED RESULT

The health request returns JSON indicating service health. The authenticated profile succeeds; deliberately missing auth produces 401 rather than silent acceptance.

WHY IT WORKED

Timeout bounds waiting; status/media/shape checks separate transport success from data-contract success; environment config prevents token hard-coding.

COMMON BEGINNER MISTAKES

- No timeout.
- Calling ``.json()`` and trusting shape blindly.
- Logging Authorization header.
- Treating 401 as retryable transient failure.

RECOVERY IF STUCK

If connection fails, verify server terminal and ``/health`` first. If 401, inspect presence of environment token without printing it. If JSON shape is wrong, fail before landing as accepted data.



C3-01-L05 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Add a client call that requires a new key from `/profile`; deliberately request a nonexistent key first and show your contract failure.

VALIDATION - DO NOT SKIP

timeout set; expected status checked; JSON/content checked; required keys checked; token absent from source/log; missing-auth case fails clearly.

ARCHITECTURE / PRODUCTION REASONING

Production clients add session pooling, request IDs, metrics and secrets managers. The basic contract remains method + boundary + timeout + status + content + schema.

TEACH IT BACK

Explain why HTTP 200 is necessary but insufficient evidence for ingestion correctness.

PROTECTED REVIEW + CHANGED RETRY

Freeze P05 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Use another mock endpoint and define its contract from scratch.

EVIDENCE / MASTERY

client + sanitized logs + successful contract + deliberate 401/shape failure + changed transfer.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L06 - API authentication, pagination and rate limits

Focused effort: 3.0 h | Matching practice P06 | Overlap: deeper engineering progression

WHY IT MATTERS

A client can return a beautiful first page and silently lose 90% of records. Completeness requires following the API continuation contract and proving the final unique set.

WHAT YOU LEARN

Keep credentials out of source; follow pagination until termination; preserve per-page evidence; handle 429 Retry-After; prove completeness and uniqueness.

PREREQUISITE / BRIDGE BOUNDARY

L05.

Deepens Bridge API use into completeness, auth hygiene and producer rate-control behavior.

PICTURE IT IN YOUR HEAD

A five-page bank statement arrives one envelope at a time. Reading page 1 successfully is not "downloaded statement." You must follow the next-page marker until the bank says there are no more pages.

NEW WORDS BEFORE WE USE THEM

Bearer token - Credential presented in Authorization header.

pagination - API pattern that returns a dataset in multiple pages.

cursor - Opaque continuation position returned by an API.

rate limit - Producer limit on request frequency/volume.

Retry-After - HTTP guidance for how long to wait after some throttling responses.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Requests Authentication

<https://requests.readthedocs.io/en/latest/user/authentication/>

Purpose: Authorization patterns

Useful segment: Authentication section

RECOMMENDED | Turtle Code - Mastering API Pagination with Python & FastAPI

https://www.youtube.com/watch?v=powvddxPs_c

Purpose: Compare limit-offset and cursor pagination, then relate to complete client extraction

Useful segment: 00:00-end; whole video is pagination

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.

C3-01-L06 - Follow with me once

FOLLOW WITH ME ONCE

- Set the fake token as environment variable.
- Fetch `/orders` page 1 and inspect its continuation field.
- Loop until continuation is absent; land each raw page with page number/cursor/run metadata.
- Combine only after raw pages are preserved; validate total IDs and uniqueness.
- Call `/rate-limit-once`, capture 429 + Retry-After, wait accordingly, then retry once.

TYPE / RUN

```
# Expected client idea, not the final P06 solution
while next_url:
    response = session.get(next_url, timeout=3)
    # validate status/content, land raw page, then follow continuation
```

EXPECTED RESULT

All mock order IDs are collected exactly once. The rate-limit endpoint shows one 429 then a successful response after respecting the server's rule.

WHY IT WORKED

Pagination termination is part of the source contract; unique-ID/full-count controls detect missed/repeated pages; throttling is handled deliberately rather than hammering the producer.

COMMON BEGINNER MISTAKES

- Stopping after first 200 response.
- Constructing guessed next URLs instead of following documented continuation.
- Retrying 429 immediately in a tight loop.
- Writing token into committed config.

RECOVERY IF STUCK

If IDs repeat, log page/cursor metadata and inspect continuation logic. If 429 repeats, inspect Retry-After and retry bounds. If token fails, diagnose auth separately from pagination.



C3-01-L06 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Change page size or mock continuation shape and prove complete extraction again.

VALIDATION - DO NOT SKIP

landed page count known; final unique IDs = expected full set; no repeated page/cursor cycle; 429 recovery bounded; token secret scan clean.

ARCHITECTURE / PRODUCTION REASONING

Real APIs may use offset, cursor, Link headers or continuation tokens. Never generalize one pagination mechanism blindly; completeness proof is universal.

TEACH IT BACK

Explain why pagination is a data-completeness contract, not a UI convenience.

PROTECTED REVIEW + CHANGED RETRY

Freeze P06 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Use a cursor-style changed mock case and re-prove complete unique extraction.

EVIDENCE / MASTERY

raw pages + page metadata + combined output + uniqueness/completeness check + 429 recovery log.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L07 - Retries, backoff, jitter and transient failures

Focused effort: 3.0 h | Matching practice P07 | Overlap: deeper engineering progression

WHY IT MATTERS

Retrying can improve reliability or amplify an outage. The difference is whether you retry only transient failures, cap attempts, spread clients with jitter and keep the operation safe to repeat.

WHAT YOU LEARN

Classify retryable/non-retryable failures; implement bounded attempts; exponential backoff + jitter; honor server guidance; prove eventual success and bounded failure.

PREREQUISITE / BRIDGE BOUNDARY

L05-L06.

Deepens ordinary exception handling into production retry policy and idempotence reasoning.

PICTURE IT IN YOUR HEAD

If a shop door is temporarily jammed, trying again after a growing pause makes sense. If your key is wrong, trying it 1,000 times only creates noise. Jitter keeps a crowd from all retrying at the same second.

NEW WORDS BEFORE WE USE THEM

transient - Failure likely to clear without changing the request, e.g. temporary 503/network issue.

backoff - Increasing delay between retry attempts.

jitter - Random variation added to retry delay to reduce synchronized retry storms.

retry budget - Maximum attempts/time allowed.

idempotence - Repeating an operation produces no unintended additional effect.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | urllib3 Retry

<https://urllib3.readthedocs.io/en/latest/reference/urllib3.util.html>

Purpose: bounded retry, backoff and jitter semantics

Useful segment: Retry parameters + backoff_jitter

RECOMMENDED | CodeLucky - Retry Strategies: Exponential Backoff & Jitter Explained

<https://www.youtube.com/watch?v=NByH-cau97A>

Purpose: See transient failure, thundering herd, backoff and jitter visually

Useful segment: 00:21-03:34

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.



C3-01-L07 - Follow with me once

FOLLOW WITH ME ONCE

- Call mock `/transient`; record first 503.
- Allow only justified statuses/transport exceptions to retry.
- Use a small capped exponential delay; add jitter; cap total attempts.
- Demonstrate endpoint eventually succeeds.
- Then lower max attempts so the same endpoint produces a clear bounded failure.

TYPE / RUN

```
delay = min(cap, base * (2 ** attempt)) + random.uniform(0, jitter)
# retry only declared transient conditions; never an infinite while True
```

EXPECTED RESULT

One run shows 503 -> 503 -> success within the budget; another deliberately small budget ends with a clear non-success exit/exception.

WHY IT WORKED

Classification prevents useless retry; exponential delay reduces pressure; jitter desynchronizes clients; bounded attempts guarantee the pipeline eventually returns control.

COMMON BEGINNER MISTAKES

- Infinite retry loop.
- Retrying 400/401/schema failures blindly.
- No jitter across many workers.
- Retrying a non-idempotent write without an idempotency strategy.

RECOVERY IF STUCK

If retries never stop, add explicit attempt/time caps first. If permanent failures are being retried, split transport/transient policy from validation/auth errors.



C3-01-L07 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Create a local function that fails twice, then succeeds. Run it through your retry helper and record exact attempt count; then test a permanent ValueError that must not retry.

VALIDATION - DO NOT SKIP

retry class explicit; max attempts explicit; delay capped; jitter present; success path and bounded-failure path both evidenced; permanent error not retried.

ARCHITECTURE / PRODUCTION REASONING

At scale, retry budgets must align with upstream rate limits, orchestration retries and downstream idempotence. Stacking multiple retry layers can multiply calls unexpectedly.

TEACH IT BACK

Explain backoff, jitter and retryability using one transient and one permanent example.

PROTECTED REVIEW + CHANGED RETRY

Freeze P07 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Change retryable status and max budget, then re-run without review open.

EVIDENCE / MASTERY

retry config/code + attempt log + success + bounded failure + non-retryable proof + changed transfer.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L08 - Database extraction and safe query boundaries

Focused effort: 2.5 h | Matching practice P08 | Overlap: deeper engineering progression

WHY IT MATTERS

A correct SQL query can still be an unsafe extraction if its window, ordering, columns and parameter handling are ambiguous.

WHAT YOU LEARN

Use explicit columns/table/view; bind values; define inclusive/exclusive window; use stable order/tie-break; reconcile to independent DB count.

PREREQUISITE / BRIDGE BOUNDARY

C2B Advanced SQL + L01. Python `sqlite3` is standard library, so no new installation.

Assume strong SQL; this lesson engineers extraction contracts, parameter binding, windows and independent source controls.

PICTURE IT IN YOUR HEAD

A warehouse picker needs a precise pick list: which shelves, which items, and exact time boundary. "Bring everything changed recently" is not an auditable extraction contract.

NEW WORDS BEFORE WE USE THEM

parameter binding - Passing values separately from SQL text so the driver handles them safely.

extraction window - Exact lower/upper boundary defining eligible records.

tie-break key - Stable secondary key when many rows share the same timestamp.

source control query - Independent query used to verify extract count/keys.

query boundary - Declared table/view, columns, filters and ordering.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Python 3.14 `sqlite3`

<https://docs.python.org/3.14/library/sqlite3.html>

Purpose: parameter binding and safe extraction boundary

Useful segment: Placeholders + execute/query examples

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.

C3-01-L08 - Follow with me once

FOLLOW WITH ME ONCE

- Open P08 `practice.db` with Python sqlite3.
- Inspect schema intentionally; choose explicit columns.
- Define lower/upper timestamps and whether each boundary is inclusive/exclusive.
- Execute parameterized query with `?` placeholders and stable `ORDER BY updated\_at, order\_id`.
- Run a separate `COUNT(\*)` query with the same eligibility boundary and compare.

TYPE / RUN

```
sql = '''SELECT order_id, updated_at, amount
FROM orders
WHERE updated_at > ? AND updated_at <= ?
ORDER BY updated_at, order_id'''
rows = conn.execute(sql, (low, high)).fetchall()
```

EXPECTED RESULT

Extracted rows are stably ordered, values were bound rather than concatenated, and independent count query equals extract row count.

WHY IT WORKED

Explicit boundary defines eligibility; parameter binding separates data from query text; stable ordering makes repeated extraction comparison deterministic.

COMMON BEGINNER MISTAKES

- `SELECT \*` as a long-lived contract.
- String-concatenating timestamps/IDs into SQL.
- Using only timestamp when ties can exist.
- Changing source query and control query together so both share same bug.

RECOVERY IF STUCK

If counts differ, print the lower/upper boundary, run source control independently, inspect equal-to-boundary rows and confirm time-zone/type semantics before widening the window.



C3-01-L08 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Extract a different window and add a tie timestamp. Prove both tied rows appear exactly once under your chosen ordering.

VALIDATION - DO NOT SKIP

explicit columns; bound values; boundary stated; stable ordering; source count equals output; key uniqueness checked.

ARCHITECTURE / PRODUCTION REASONING

Real sources add snapshots/transactions, load impact and permissions. Keep extraction read scope minimal and coordinate with source owners; correctness is only one production concern.

TEACH IT BACK

Explain why parameter binding and deterministic extraction windows solve different problems.

PROTECTED REVIEW + CHANGED RETRY

Freeze P08 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Use a new window/table and rebuild query + independent control from blank.

EVIDENCE / MASTERY

SQL/Python + bound values + output + source control + tie-boundary proof + changed transfer.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L09 - Historical batch loading and landing metadata

Focused effort: 2.5 h | Matching practice P09 | Overlap: deeper engineering progression

WHY IT MATTERS

Historical loads are the evidence base for later transformations. Overwriting `latest.csv` destroys traceability and makes backfills/reprocessing hard to reproduce.

WHAT YOU LEARN

Land each historical batch immutably; record run/window/source identity/hash/count; build a manifest index; distinguish event window from ingestion run time.

PREREQUISITE / BRIDGE BOUNDARY

L01 and L08 concepts.

New engineering implementation depth.

PICTURE IT IN YOUR HEAD

Archive boxes are labeled by what period they contain and when they were received. You do not throw away May when June arrives; you catalogue both.

NEW WORDS BEFORE WE USE THEM

historical load - Loading a past bounded data period.

batch manifest - Per-batch metadata record.

source window - Business/source period represented by the batch.

ingested_at - When your system received/processed it.

immutable path - Path whose historical contents are not silently replaced.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Course-built landing manifest pattern

Local lesson book

Purpose: Immutable historical landing + per-batch metadata

Useful segment: Whole lesson

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.



C3-01-L09 - Follow with me once

FOLLOW WITH ME ONCE

- Use P09 May/June/July files.
- Create a unique landing directory for each source object/window.
- Compute sha256 + bytes + row count for each raw file.
- Write per-batch manifest.json and a manifest index.
- Rerun one unchanged batch and make the policy explicit: detect identical fingerprint and avoid destructive overwrite.

TYPE / RUN

```
landing/<source>/<window>/<run_id>/raw.csv  
landing/<source>/<window>/<run_id>/manifest.json
```

EXPECTED RESULT

Three historical raw objects remain addressable with distinct manifests. An unchanged rerun is recognized rather than replacing prior evidence.

WHY IT WORKED

Window identity answers what data the batch represents; run ID answers when/how you processed it; fingerprint proves whether source bytes changed.

COMMON BEGINNER MISTAKES

- One `latest.csv` path.
- Using filesystem modification time as business event time.
- No hash or count.
- Treating changed historical bytes as normal rerun without alert/evidence.

RECOVERY IF STUCK

If prior batches were overwritten, reconstruct from source if available. From now on use immutable paths and record the evidence gap rather than inventing old fingerprints.



C3-01-L09 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Add an August batch and then modify one source row. Demonstrate your landing logic distinguishes identical rerun from changed-source rerun.

VALIDATION - DO NOT SKIP

every batch has unique path + manifest + hash + count + source window; changed bytes yield changed fingerprint; original remains preserved.

ARCHITECTURE / PRODUCTION REASONING

At cloud scale, manifests can live in metadata tables/catalogs and raw objects in versioned storage. The audit properties remain identical.

TEACH IT BACK

Explain the difference between source window, run ID, ingested_at and file modification time.

PROTECTED REVIEW + CHANGED RETRY

Freeze P09 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Land a changed-source historical batch under a new run and explain why it must not overwrite old evidence.

EVIDENCE / MASTERY

landing tree + manifests/index + hash/count controls + rerun behavior + changed-source evidence.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L10 - Incremental loads and high-water marks

Focused effort: 3.5 h | Matching practice P10 | Overlap: deeper engineering progression

WHY IT MATTERS

A timestamp-only watermark can skip records when multiple rows share the timestamp or when state advances before durable publication. Incremental correctness is primarily a state/commit problem.

WHAT YOU LEARN

Use a deterministic tuple cursor; define old/new high-water; select new eligibility; publish durably; advance state last; test ties and failure before commit.

PREREQUISITE / BRIDGE BOUNDARY

L08-L09.

New core engineering state-management skill.

PICTURE IT IN YOUR HEAD

A bookmark that records only page number fails if two notes sit on the same page. Record page + line number. And do not move the bookmark until you have actually saved what you read.

NEW WORDS BEFORE WE USE THEM

high-water mark - Durable boundary representing the last committed source position.

tuple cursor - Ordered boundary such as (updated_at, order_id) to break ties.

state commit - Durable update of cursor only after output is safe.

incremental load - Process only records beyond previously committed boundary.

atomic replace - Swap new state into place so readers see old or new complete state, not half-written state.

USEFUL VIDEO / SOURCE

RECOMMENDED | Microsoft - Incremental copy using watermark

<https://learn.microsoft.com/en-us/azure/data-factory/tutorial-incremental-copy-overview>

Purpose: old/new watermark pattern and bounded incremental extraction

Useful segment: High-level solution + watermark steps

RECOMMENDED | Cloudpandith - ADF Incremental Load Using Watermark Table

https://www.youtube.com/watch?v=U86eFO_1oeU

Purpose: Visualize watermark/delta-load architecture; transfer concept back to vendor-neutral tuple cursor

Useful segment: 00:00-end; whole lesson is incremental/watermark

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.

C3-01-L10 - Follow with me once

FOLLOW WITH ME ONCE

- Inspect P10 `state\_initial.json`, run1.csv and run2.csv.
- Represent cursor as `(updated\_at, order\_id)`, not timestamp alone.
- Filter records lexicographically beyond old cursor; sort by the same tuple.
- Write/validate durable output first.
- Only after validation succeeds, write temporary state and atomically replace checkpoint. Inject a failure before this step once.

TYPE / RUN

```
cursor = (state['updated_at'], state['order_id'])
eligible = [r for r in rows if (r['updated_at'], r['order_id']) > cursor]
# publish + validate first
# atomic state replace LAST
```

EXPECTED RESULT

Rows sharing a timestamp are not skipped. A failure before state commit leaves old state unchanged; rerun replays safely and then advances once.

WHY IT WORKED

Tuple ordering gives a total deterministic boundary across ties. Committing state after durable validation prevents the cursor from jumping past unpublished data.

COMMON BEGINNER MISTAKES

- Timestamp-only cursor.
- Advancing state before output validation.
- Using current clock time as source watermark.
- No proof of what happens on replay.

RECOVERY IF STUCK

If records were skipped, compare old state with source rows around the boundary. Restore last known safe committed state, replay from there, deduplicate by declared key, and document recovery.



C3-01-L10 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Add two rows with identical updated_at and distinct IDs. Inject failure after output temp file but before state replace; rerun and prove no skip/duplicate accepted state.

VALIDATION - DO NOT SKIP

old/new cursor logged; every eligible tuple > old and <= new; source/output counts reconcile; state unchanged on failure; rerun stable.

ARCHITECTURE / PRODUCTION REASONING

Large systems may use log positions, sequence numbers or orchestration metadata, but "publish durably, validate, commit state last" remains a durable design principle.

TEACH IT BACK

Explain why `(timestamp, id)` is safer than timestamp alone and why checkpoint commit order matters.

PROTECTED REVIEW + CHANGED RETRY

Freeze P10 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Use a different tie key and changed two-run dataset; prove same invariants.

EVIDENCE / MASTERY

state before/after + selected tuples + durable output + reconciliation + injected failure + replay proof.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L11 - CDC concepts and change-event semantics

Focused effort: 3.5 h | Matching practice P11 | Overlap: deeper engineering progression

WHY IT MATTERS

Incremental polling asks "what rows look new now?" CDC preserves a stream of database changes. Without operation/key/source-position semantics, replaying updates/deletes can corrupt current state.

WHAT YOU LEARN

Interpret create/update/delete/read events; preserve key/source position; apply events deterministically to current state; retain raw event history; reason about ordering/replay.

PREREQUISITE / BRIDGE BOUNDARY

L10. No Kafka installation here: C3-06 handles streaming systems later.

New engineering concept. This lesson teaches CDC event semantics without prematurely installing Kafka/Debezium.

PICTURE IT IN YOUR HEAD

Instead of receiving a fresh address book, you receive signed change slips: ADD Ravi, CHANGE Ravi address, DELETE Ada. To rebuild the address book, you need each slip's person key and reliable order.

NEW WORDS BEFORE WE USE THEM

CDC - Change Data Capture: representing database changes as events.

operation code - Meaning such as create/update/delete/read snapshot.

before/after - Record images around a change when available.

source position - Database-log position/offset used for ordering/resume.

tombstone/delete - Event semantics indicating record removal; exact format depends on platform.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Debezium MySQL Connector - stable

<https://debezium.io/documentation/reference/stable/connectors/mysql.html>

Purpose: CDC event semantics, source position, key/value and operations

Useful segment: Change events + event structure

RECOMMENDED | Binod Suman Academy - Change Data Capture (CDC): Why & How

https://www.youtube.com/watch?v=dN_11nBcv_A

Purpose: Visual CDC/system-design companion before applying event semantics

Useful segment: 00:00-end; whole video is CDC concept/system design

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.

C3-01-L11 - Follow with me once

FOLLOW WITH ME ONCE

- Open P11 `cdc\_events.json`; do not install Debezium/Kafka.
- For every event identify key, op, source position and before/after data.
- Apply events in source-position order to a tiny in-memory current-state dictionary.
- For delete, remove/mark the key according to declared target contract.
- Replay the same raw event list and prove the rebuilt current state is deterministic.

TYPE / RUN

```
# Conceptual reducer
if op in {'c','r','u'}: current[key] = event['after']
elif op == 'd': current.pop(key, None)
```

EXPECTED RESULT

The final current-state keys/values match the ordered event semantics. Raw CDC events remain preserved even though current state contains only the latest state.

WHY IT WORKED

Source position defines event order; key identifies target row; op determines state transition. Keeping raw events separate lets you replay/audit.

COMMON BEGINNER MISTAKES

- Treating delete like an update with null fields without contract.
- Deduplicating different updates just because key repeats.
- Ignoring source ordering.
- Installing Kafka now even though it is not needed for event-semantic mastery.

RECOVERY IF STUCK

If final state looks wrong, replay one event at a time and log position/key/op/current state. Find first divergence instead of patching the final table.



C3-01-L11 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Add a create -> update -> delete -> recreate sequence for one key and predict final state before running.

VALIDATION - DO NOT SKIP

positions ordered/unique as required; each op handled; final state manually predicted; replay yields identical final state; raw log unchanged.

ARCHITECTURE / PRODUCTION REASONING

Debezium reads database logs and emits richer envelopes. Streaming delivery adds offsets/checkpoints later, but this lesson isolates the semantic core first.

TEACH IT BACK

Explain the difference between polling by updated_at and CDC, and why repeated keys in CDC are normally meaningful.

PROTECTED REVIEW + CHANGED RETRY

Freeze P11 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Use a changed event sequence including delete/recreate; rebuild from raw events without review.

EVIDENCE / MASTERY

event interpretation table + reducer output + final-state control + replay proof + changed event transfer.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L12 - Schema drift, compatibility, temporal/time-zone correctness and quarantine

Focused effort: 3.5 h | Matching practice P12 | Overlap: deeper engineering progression

WHY IT MATTERS

A new column may be safe while a renamed key, changed type or timezone-naive timestamp may make the same pipeline silently wrong. Engineers classify changes; they do not guess.

WHAT YOU LEARN

Detect added/removed/renamed/type changes; define compatible/unsafe policy; distinguish naive/aware time; normalize trusted timestamps to UTC; quarantine ambiguity; block state advance on critical drift.

PREREQUISITE / BRIDGE BOUNDARY

L02-L05 and L10-L11 concepts.

Deepens Bridge quality controls into evolution policy and temporal correctness at ingestion.

PICTURE IT IN YOUR HEAD

A delivery form gains an optional "note" box: probably safe. But if "customer_id" becomes "client_code," guessing they are identical could join the wrong people. Timezones are similar: "09:00" without location is incomplete information.

NEW WORDS BEFORE WE USE THEM

schema drift - Source schema changing over time.

compatibility policy - Declared rule for which changes may continue automatically.

timezone-aware - Timestamp includes enough timezone/offset context to map to UTC.

naive timestamp - Datetime without timezone/offset context.

quarantine reason - Machine/human-readable explanation for rejecting/holding ambiguous input.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Python 3.14 datetime

<https://docs.python.org/3.14/library/datetime.html>

Purpose: aware vs naive datetimes and UTC conversion

Useful segment: Aware/naive objects + timezone

REQUIRED REFERENCE | Course-built schema compatibility policy

Local lesson assets

Purpose: Compatible vs unsafe drift + quarantine

Useful segment: Whole lesson

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.

C3-01-L12 - Follow with me once

FOLLOW WITH ME ONCE

- Compare P12 day1/day2 headers and `compatibility\_policy.json`.
- Classify each change: added optional, removed required, rename/semantic change, type change.
- Parse timestamps; accept aware timestamps and normalize to UTC.
- For naive timestamp, use source contract timezone only if explicitly declared; otherwise quarantine.
- If required key/type/time semantics are unresolved, fail validation and leave checkpoint unchanged.

TYPE / RUN

```
from datetime import datetime, timezone
dt=datetime.fromisoformat('2026-09-10T09:00:00+05:30')
print(dt.astimezone(timezone.utc).isoformat())
```

EXPECTED RESULT

The aware +05:30 timestamp prints an equivalent UTC instant. A naive timestamp is not silently treated as local or UTC unless the contract declares that rule.

WHY IT WORKED

Schema policy separates structural change from semantic guesswork; aware timestamps map to an instant; quarantine preserves ambiguous records without inventing meaning.

COMMON BEGINNER MISTAKES

- Automatically accepting any added/removed/renamed field.
- Using machine local timezone implicitly.
- Replacing bad timestamp with current time.
- Advancing state after critical drift blocked records unpredictably.

RECOVERY IF STUCK

Stop publication/state advance; preserve raw batch; record observed schema/time ambiguity; get producer/contract clarification; update versioned policy; replay from unchanged checkpoint.



C3-01-L12 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Create day3 with one compatible nullable field and one critical type change. Show the first can proceed while the critical record/batch follows your declared stop/quarantine rule.

VALIDATION - DO NOT SKIP

schema diff recorded; policy decision explicit; UTC conversion proved; ambiguous time retained in quarantine; critical unresolved drift prevents unsafe state advance.

ARCHITECTURE / PRODUCTION REASONING

Schema registries/contracts can automate compatibility checks, but human semantic ownership is still required for renames and business meaning.

TEACH IT BACK

Explain why a timezone-naïve timestamp and a renamed key are both examples of incomplete semantic information.

PROTECTED REVIEW + CHANGED RETRY

Freeze P12 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Use a changed schema/time case; classify it without copying prior decisions.

EVIDENCE / MASTERY

schema diff + policy + accepted/quarantine + UTC proof + state decision + changed case explanation.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L13 - Checkpoint/state management and replay safety

Focused effort: 3.5 h | Matching practice P13 | Overlap: deeper engineering progression

WHY IT MATTERS

Failures happen between steps. If your checkpoint says data is processed before durable output is trustworthy, a restart can permanently skip data.

WHAT YOU LEARN

Implement state lifecycle; temp write + atomic replace; distinguish processing from committed state; deliberately fail before commit; make outputs replay-safe.

PREREQUISITE / BRIDGE BOUNDARY

L10-L12.

New engineering reliability skill.

PICTURE IT IN YOUR HEAD

You are checking off boxes during a move. Never mark "box delivered" while it is still in the truck. If the truck breaks down, the unchecked box gets retried.

NEW WORDS BEFORE WE USE THEM

checkpoint - Durable resume position/state.

commit point - Moment after which the system considers work safely complete.

replay - Processing previously seen input again after restart/recovery.

atomic replace - Replacing state as one filesystem operation so partial JSON is not exposed.

side effect - External change such as inserted rows/files/messages that may duplicate on replay.

USEFUL VIDEO / SOURCE

REQUIRED REFERENCE | Course-built durable state pattern

Local lesson book

Purpose: write/validate/publish then atomic state replace

Useful segment: Whole lesson

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.



C3-01-L13 - Follow with me once

FOLLOW WITH ME ONCE

- Read old checkpoint.
- Select input beyond checkpoint.
- Land raw + stage + validate.
- Publish accepted output durably and idempotently.
- Write new state to temporary file then `os.replace` only after validation. Add a deliberate exception immediately before state replace; rerun.

TYPE / RUN

```
tmp = state_path.with_suffix('.tmp')
tmp.write_text(json.dumps(new_state))
# only after output validation
os.replace(tmp, state_path)
```

EXPECTED RESULT

Injected failure leaves old checkpoint intact. Rerun may repeat work, but accepted/current output does not gain unintended duplicates, then state advances once.

WHY IT WORKED

The old checkpoint remains the source of resume truth until durable output is proven. Atomic replacement avoids half-written checkpoint files.

COMMON BEGINNER MISTAKES

- Checkpoint first, output second.
- Appending duplicate outputs on every replay.
- Catching exception then returning success.
- Deleting old state without backup/recovery evidence.

RECOVERY IF STUCK

Restore last known committed checkpoint, inspect partial side effects, make write idempotent/deduplicated, replay from safe boundary, validate, then commit state.



C3-01-L13 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Use P13 changed batch. Fail once after staging but before state commit. Compare hashes/keys after rerun and prove the committed result is stable.

VALIDATION - DO NOT SKIP

state_before captured; state unchanged on injected failure; rerun accepted keys stable; temp state not left as active checkpoint; final state equals last durable accepted boundary.

ARCHITECTURE / PRODUCTION REASONING

Databases/object stores/orchestrators offer different transaction primitives. You still need to identify your real commit point and replay behavior explicitly.

TEACH IT BACK

Explain why "at least once processing" can be safe when outputs are replay-safe and state commits last.

PROTECTED REVIEW + CHANGED RETRY

Freeze P13 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Move the injected failure to a different pre-commit point and re-prove safety.

EVIDENCE / MASTERY

state snapshots + failure log + partial-output inspection + replay result + uniqueness/hash controls + changed retry.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.



C3-01-L14 - Ingestion testing, reconciliation and backfills

Focused effort: 3.5 h | Matching practice P14 | Overlap: deeper engineering progression

WHY IT MATTERS

A job can be green and wrong. Backfills can also corrupt the live incremental cursor if historical replay shares state carelessly.

WHAT YOU LEARN

Test parser/contracts/state failures; reconcile source-to-output; design isolated backfill windows/state; prove backfill does not mutate live cursor; document operator evidence.

PREREQUISITE / BRIDGE BOUNDARY

L01-L13: pytest already introduced in C3-00.

Builds on B02 testing/automation and B04 data quality, but now tests ingestion boundaries, state transitions, replay and backfill isolation.

PICTURE IT IN YOUR HEAD

Live ingestion is today's train schedule; a backfill is a special historical train. It may use the same tracks but must not move today's station clock backward or forward.

NEW WORDS BEFORE WE USE THEM

reconciliation - Independent controls that account for source vs accepted/quarantine/output.

backfill - Intentional processing of a historical period.

live state - Checkpoint used by normal ongoing incremental process.

fixture - Controlled test input used to reproduce behavior.

invariant - Condition that must remain true across runs, e.g. source = accepted + quarantine.

USEFUL VIDEO / SOURCE

RECOMMENDED | pytest documentation

<https://docs.pytest.org/en/stable/>

Purpose: boundary/state testing

Useful segment: Getting started + assertions

REQUIRED REFERENCE | Course-built reconciliation/backfill pattern

Local lesson book

Purpose: source controls, isolated backfill state, production reasoning

Useful segment: Whole lesson

Video rule: visuals help you picture the concept, but the book + practice remains sufficient if a video is unavailable.



C3-01-L14 - Follow with me once

FOLLOW WITH ME ONCE

- Run `pytest -q` in a small ingestion exercise.
- Write a test for one valid parse, one quarantine/failure, one cursor tie and one replay invariant.
- Compute source = accepted + quarantine and unique-key controls.
- Run P14 historical backfill with explicit start/end and a separate backfill state/output namespace.
- Hash/read `live\_state.json` before and after; prove backfill did not mutate it.

TYPE / RUN

```
before = hashlib.sha256(Path('live_state.json').read_bytes()).hexdigest()
# run bounded backfill using separate state/output
after = hashlib.sha256(Path('live_state.json').read_bytes()).hexdigest()
assert before == after
```

EXPECTED RESULT

Tests pass only after intended behavior is implemented. Backfill processes its bounded history while the live-state hash remains unchanged.

WHY IT WORKED

Tests guard known behavior; reconciliation checks data accounting independently; state isolation prevents a historical operation from moving the live resume boundary.

COMMON BEGINNER MISTAKES

- Only happy-path tests.
- Asserting output produced by the same buggy function without independent control.
- Backfill reusing live cursor file.
- Calling green exit code "data validated."

RECOVERY IF STUCK

When test/control fails, freeze source + state, find first violated invariant, repair that boundary, rerun changed fixture; never "fix" expected values to match bad output.



C3-01-L14 - Transfer, validate and master

INDEPENDENT SMALL VERSION

Backfill a different historical date window and add one malformed record. Prove bounded eligibility, quarantine accounting and unchanged live state.

VALIDATION - DO NOT SKIP

pytest includes failure/edge cases; source accounting reconciles; keys/temporal bounds checked; live state hash unchanged; backfill state clearly separate.

ARCHITECTURE / PRODUCTION REASONING

Production backfills need capacity/rate-limit controls and downstream coordination. Treat them as deliberate deployments with explicit bounds, observability and rollback/replay plans.

TEACH IT BACK

Explain why testing, reconciliation and backfill isolation are three different safeguards.

PROTECTED REVIEW + CHANGED RETRY

Freeze P14 first. Only then open the matching C3-022 review section. Close it and complete this changed retry: Run a changed historical window with one deliberate invalid record and re-prove all invariants.

EVIDENCE / MASTERY

tests + reconciliation + backfill bounds + separate state + live-state hash proof + README/runbook + changed transfer.

MOVE-ON RULE

Master only when the changed version succeeds without the review open, the independent controls pass, and you can explain the failure/recovery path. A command that merely runs is not mastery.

Mastery checklist + revision quick reference

- ☐ Every lesson: guided rebuild completed, genuine independent attempt saved, validation recorded.
- ☐ Protected review opened only after the attempt; changed retry solved after closing the review.
- ☐ Explain-back: business/engineering purpose, grain or state, one failure mode, one recovery step, one validation check.
- ☐ Mini-Lab / integrated drill completed with reproducible evidence.
- ☐ Gate A attempted independently. If NEEDS_REPAIR: review viewed, targeted repair completed, changed micro-proof recorded, then a fresh B/C assigned explicitly.
- ☐ No unresolved critical failure, secret/private data leakage, fabricated live-run evidence or copied Gate solution.
- ☐ Evidence is saved with enough context for another person to reproduce or inspect it.

Revision quick reference

Wrong result / wrong grain	Return to validation + data contract / grain lesson; reproduce smallest failing case.
Works once, rerun breaks	Return to idempotence/state/checkpoint/recovery lesson; prove clean replay.
Cannot explain failure	Return to mental model + break/fix section; write the failure boundary in your own words.
Gate NEEDS_REPAIR	Open only its protected review, repair named weak competency, solve changed micro-proof, then assign fresh retest.
Tool/vendor changed	Use the current official source for syntax/UI, but keep the book's durable engineering contract and validation.